EXPERIMENT - 09

Aim

To design input forms that validate user data such as email and phone number and display appropriate error messages using HTML, CSS, and JavaScript (Validator.js).

Introduction

Form validation is an important part of web development that ensures users enter correct and valid data. It helps prevent errors and improves user experience.

In this experiment, an input form is created to validate email and phone number fields. JavaScript along with Validator.js is used to check inputs and display error messages for invalid data.

Procedure :

Step 1: Create HTML Form

register.html > ...

```html
<!DOCTYPE html>
<html>
<head>
  <title>Registration Form</title>
  <link rel="stylesheet" href="style.css">
</head>

<body>
<div class="container">
<form id="form">

<h2>Register</h2>

<input type="text" id="name" placeholder="Enter Your Name">
<span id="nameError" class="error"></span>

<input type="email" id="email" placeholder="Enter Your Email">
<span id="emailError" class="error"></span>

<input type="text" id="phone" placeholder="Enter Your Phone">
<span id="phoneError" class="error"></span>

<input type="password" id="password" placeholder="Enter Your Password">
<span id="passError" class="error"></span>

<button type="submit">Register</button>

</form>
</div>

<script src="https://cdnjs.cloudflare.com/ajax/libs/validator/13.6.0/validator.min.js"></script>
<script src="script.js"></script>
</body>
</html>
```

- Designed a form with input fields for email and phone number
- Added labels and input boxes
- Included error message placeholders using
- Added submit button

Step 2: Apply CSS Styling

```
# style.css > ...
body {
  font-family: Arial;
  background: linear-gradient(to right, #3b7ab0, #10c0c9);
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
}

.container {
  background: white;
  padding: 25px;
  border-radius: 8px;
  width: 250px;
}

input {
  width: 100%;
  margin: 8px 0;
  padding: 8px;
}

button {
  width: 100%;
  padding: 10px;
  background: #024ff4;
  color: white;
  border: none;
}

.error {
  color: red;
  font-size: 12px;
}

```

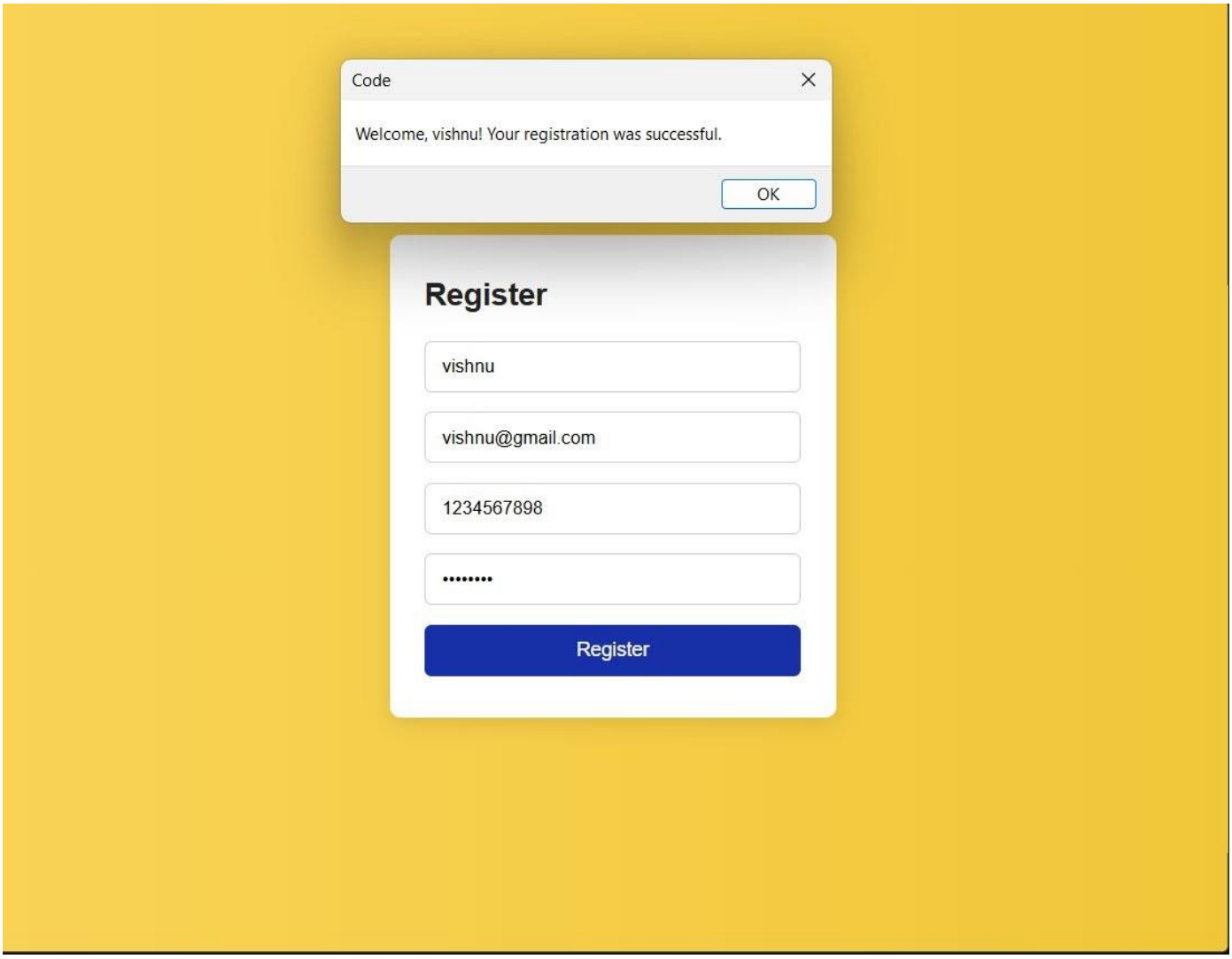
- Styled the form using CSS
- Added background color, padding, and borders
- Improved layout using alignment and spacing
- Styled error messages in red color

Step 3: Add JavaScript Validation

```js
JS script.js > ...
document.getElementById("form").addEventListener("submit", function(e) {
  e.preventDefault();
  let name = document.getElementById("name").value;
  let email = document.getElementById("email").value;
  let phone = document.getElementById("phone").value;
  let password = document.getElementById("password").value;

  let nameError = document.getElementById("nameError");
  let emailError = document.getElementById("emailError");
  let phoneError = document.getElementById("phoneError");
  let passError = document.getElementById("passError");
  nameError.textContent = "";
  emailError.textContent = "";
  phoneError.textContent = "";
  passError.textContent = "";
  if (name === "") {
    nameError.textContent = "Please Enter Your Name!";
  }
  if (!validator.isEmail(email)) {
    emailError.textContent = "Invalid email Address!";
  }
  if (!validator.isMobilePhone(phone, "any")) {
    phoneError.textContent = "Invalid phone Number!";
  }
  if (!validator.isStrongPassword(password)) {
    passError.textContent = "Weak password (use uppercase, number, symbol)";
  }
  if (
    name !== "" &&
    validator.isEmail(email) &&
    validator.isMobilePhone(phone, "any") &&
    validator.isStrongPassword(password)
  ) {
    alert("Registration Successful!");
  }
});
```

- Used JavaScript to validate input fields
- Integrated Validator.js library
- Checked:
  - Valid email format
  - Valid phone number
- Displayed error messages if inputs are incorrect

Step 4: Test the Form

- Entered valid and invalid inputs
- Verified correct error messages are shown
- Ensured form works properly for all cases